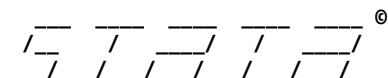


Statistics/Data analysis

User: Time series project



17.0
MP-Parallel Edition

Statistics and Data Science

Copyright 1985-2021 StataCorp LLC
StataCorp
4905 Lakeway Drive
College Station, Texas 77845 USA
800-STATA-PC <https://www.stata.com>
979-696-4600 stata@stata.com

Stata license: Unlimited-user 64-core network perpetual
Serial number: 18461036
Licensed to: TEAM BTCR
TEAM BTCR

Notes:

1. Unicode is supported; see [help unicode advice](#).
2. More than 2 billion observations are allowed; see [help obs advice](#).
3. Maximum number of variables is set to 5,000; see [help set maxvar](#).

```
1 . import excel "C:\Users\Abdul Rehman\Documents\gold_data_daily_comprehensive_cleaned.xlsx", sheet("gold_data_daily_comp
(20 vars, 6,019 obs)

2 . browse

3 . generate stata_date = date(date, "DMY")
date not found
r(111);

4 . generate stata_date = Date(date, "DMY")
unknown function Date()
r(133);

5 . generate stata_date = Date(Date, "DMY")
unknown function Date()
r(133);

6 . generate stata_date = date(Date, "DMY")
type mismatch
r(109);

7 .
8 .
9 . generate stata_date = date(date, "DMY")
date not found
r(111);

10 .
11 . format stata_date %td
variable stata_date not found
r(111);

12 . generate stata_date = date(Date, "DMY")
type mismatch
r(109);
```

```

13 . generate stata_date = mdy(Month, Day, Year)

14 . format stata_date %td

15 . bscal cdate = stata_date
command bscal is unrecognized
r(199);

16 . bcal create goldcal, from(stata_date) generate(cdate) replace
file goldcal.stbcal could not be opened for output
r(603);

17 . bcal cdate = stata_date
cdate invalid bcal subcommand
r(198);

18 . generate time_index = _n

19 . tsset time_index

Time variable: time_index, 1 to 6019
Delta: 1 unit

20 . generate ln_price = log(close)
close not found
r(111);

21 . generate ln_price = log(Close)

22 . generate log_return = d.ln_price
(1 missing value generated)

23 . dfuller ln_price, trend lags(4)

```

```
25 . arima ln_price, arima(1,1,1) vce(robust)
```

```
(setting optimization to BHHH)
```

```
Iteration 0: log pseudolikelihood = 18466.982
Iteration 1: log pseudolikelihood = 18469.471
Iteration 2: log pseudolikelihood = 18469.516
Iteration 3: log pseudolikelihood = 18469.517
Iteration 4: log pseudolikelihood = 18469.517
```

```
(switching optimization to BFGS)
```

```
Iteration 5: log pseudolikelihood = 18469.517
Iteration 6: log pseudolikelihood = 18469.517
Iteration 7: log pseudolikelihood = 18469.517
Iteration 8: log pseudolikelihood = 18469.517
```

```
ARIMA regression
```

```
Sample: 2 thru 6019
```

```
Number of obs = 6018
```

```
Wald chi2(2) = 1079.20
```

```
Log pseudolikelihood = 18469.52
```

```
Prob > chi2 = 0.0000
```

D.ln_price	Semirobust		z	P> z	[95% conf. interval]	
	Coefficient	std. err.				
ln_price						
_cons	.0004693	.0001206	3.89	0.000	.0002329	.0007057
ARMA						
ar						
L1.	.933507	.0358009	26.07	0.000	.8633385	1.003676
ma						
L1.	-.9448296	.0321546	-29.38	0.000	-1.007851	-.8818078
/sigma	.0112432	.0002149	52.32	0.000	.010822	.0116645

Note: The test of the variance against zero is one sided, and the two-sided confidence interval is truncated at zero.

```
26 . estimates store gold_arima
```

```
27 . tsappend, add(30)
```

```
28 . predict ln_forecast, y dynamic(tssample() == 0)
    unknown function tssample()
    r(133);
```

```
29 . generate price_forecast = exp(ln_forecast)
    ln_forecast not found
    r(111);
```

```
30 . predict ln_forecast, y dynamic(tssample() == 0)
    unknown function tssample()
    r(133);
```

```

31 . generate historic_data = 1

32 . tsappend, add(30)

33 . predict ln_forecast, y dynamic(historic_data == .)
    (6,079 missing values generated)

34 . generate price_forecast = exp(ln_forecast)
    (6,079 missing values generated)

35 . list cdate price_forecast if missing(close)
    variable cdate not found
    r(111);

36 . list time_index price_forecast if missing(close)
    close not found
    r(111);

37 . list time_index price_forecast if missing(Close)

```

	time_i~x	price_~t
6020.	6020	.
6021.	6021	.
6022.	6022	.
6023.	6023	.
6024.	6024	.
6025.	6025	.
6026.	6026	.
6027.	6027	.
6028.	6028	.
6029.	6029	.
6030.	6030	.
6031.	6031	.
6032.	6032	.
6033.	6033	.
6034.	6034	.
6035.	6035	.
6036.	6036	.
6037.	6037	.
6038.	6038	.
6039.	6039	.
6040.	6040	.
6041.	6041	.
6042.	6042	.
6043.	6043	.
6044.	6044	.
6045.	6045	.
6046.	6046	.
6047.	6047	.
6048.	6048	.
6049.	6049	.
6050.	6050	.
6051.	6051	.
6052.	6052	.
6053.	6053	.
6054.	6054	.
6055.	6055	.
6056.	6056	.

6057.	6057	.
6058.	6058	.
6059.	6059	.
6060.	6060	.
6061.	6061	.
6062.	6062	.
6063.	6063	.
6064.	6064	.
6065.	6065	.
6066.	6066	.
6067.	6067	.
6068.	6068	.
6069.	6069	.
6070.	6070	.
6071.	6071	.
6072.	6072	.
6073.	6073	.
6074.	6074	.
6075.	6075	.
6076.	6076	.
6077.	6077	.
6078.	6078	.
6079.	6079	.

```
38 . scalar last_obs = _N
```

```
39 . tsappend, add(30)
```

```
40 . predict ln_forecast, y dynamic(last_obs + 1)
    variable ln_forecast already defined
    r(110);
```

```
41 . list time_index price_forecast if missing(Close)
```

	time_i~x	price_~t
6020.	6020	.
6021.	6021	.
6022.	6022	.
6023.	6023	.
6024.	6024	.
6025.	6025	.
6026.	6026	.
6027.	6027	.
6028.	6028	.
6029.	6029	.
6030.	6030	.
6031.	6031	.
6032.	6032	.
6033.	6033	.
6034.	6034	.
6035.	6035	.
6036.	6036	.
6037.	6037	.
6038.	6038	.
6039.	6039	.
6040.	6040	.

6041.	6041	.
6042.	6042	.
6043.	6043	.
6044.	6044	.
6045.	6045	.
6046.	6046	.
6047.	6047	.
6048.	6048	.
6049.	6049	.
6050.	6050	.
6051.	6051	.
6052.	6052	.
6053.	6053	.
6054.	6054	.
6055.	6055	.
6056.	6056	.
6057.	6057	.
6058.	6058	.
6059.	6059	.
6060.	6060	.
6061.	6061	.
6062.	6062	.
6063.	6063	.
6064.	6064	.
6065.	6065	.
6066.	6066	.
6067.	6067	.
6068.	6068	.
6069.	6069	.
6070.	6070	.
6071.	6071	.
6072.	6072	.
6073.	6073	.
6074.	6074	.
6075.	6075	.
6076.	6076	.
6077.	6077	.
6078.	6078	.
6079.	6079	.
6080.	6080	.
6081.	6081	.
6082.	6082	.
6083.	6083	.
6084.	6084	.
6085.	6085	.
6086.	6086	.
6087.	6087	.
6088.	6088	.
6089.	6089	.
6090.	6090	.
6091.	6091	.
6092.	6092	.
6093.	6093	.
6094.	6094	.
6095.	6095	.
6096.	6096	.

6097.	6097	.
6098.	6098	.
6099.	6099	.
6100.	6100	.
6101.	6101	.
6102.	6102	.
6103.	6103	.
6104.	6104	.
6105.	6105	.
6106.	6106	.
6107.	6107	.
6108.	6108	.
6109.	6109	.

```

42 . predict ln_forecast, y dynamic(last_obs + 1)
    variable ln_forecast already defined
    r(110);

43 . scalar last_obs = _N

44 . capture drop ln_forecast

45 .
46 . capture drop price_forecast

47 . capture tsappend, add(30)

48 . predict ln_forecast, y dynamic(last_obs + 1)
    (120 missing values generated)

49 . generate price_forecast = exp(ln_forecast)
    (120 missing values generated)

50 . list time_index price_forecast if missing(Close)

```

	time_i~x	price_~t
6020.	6020	4978.872
6021.	6021	.
6022.	6022	.
6023.	6023	.
6024.	6024	.
6025.	6025	.
6026.	6026	.
6027.	6027	.
6028.	6028	.
6029.	6029	.
6030.	6030	.
6031.	6031	.
6032.	6032	.
6033.	6033	.
6034.	6034	.
6035.	6035	.
6036.	6036	.
6037.	6037	.
6038.	6038	.
6039.	6039	.
6040.	6040	.
6041.	6041	.

6042.	6042	.
6043.	6043	.
6044.	6044	.
6045.	6045	.
6046.	6046	.
6047.	6047	.
6048.	6048	.
6049.	6049	.
6050.	6050	.
6051.	6051	.
6052.	6052	.
6053.	6053	.
6054.	6054	.
6055.	6055	.
6056.	6056	.
6057.	6057	.
6058.	6058	.
6059.	6059	.
6060.	6060	.
6061.	6061	.
6062.	6062	.
6063.	6063	.
6064.	6064	.
6065.	6065	.
6066.	6066	.
6067.	6067	.
6068.	6068	.
6069.	6069	.
6070.	6070	.
6071.	6071	.
6072.	6072	.
6073.	6073	.
6074.	6074	.
6075.	6075	.
6076.	6076	.
6077.	6077	.
6078.	6078	.
6079.	6079	.
6080.	6080	.
6081.	6081	.
6082.	6082	.
6083.	6083	.
6084.	6084	.
6085.	6085	.
6086.	6086	.
6087.	6087	.
6088.	6088	.
6089.	6089	.
6090.	6090	.
6091.	6091	.
6092.	6092	.
6093.	6093	.
6094.	6094	.
6095.	6095	.
6096.	6096	.
6097.	6097	.

6098.	6098	.
6099.	6099	.
6100.	6100	.
6101.	6101	.
6102.	6102	.
6103.	6103	.
6104.	6104	.
6105.	6105	.
6106.	6106	.
6107.	6107	.
6108.	6108	.
6109.	6109	.
6110.	6110	.
6111.	6111	.
6112.	6112	.
6113.	6113	.
6114.	6114	.
6115.	6115	.
6116.	6116	.
6117.	6117	.
6118.	6118	.
6119.	6119	.
6120.	6120	.
6121.	6121	.
6122.	6122	.
6123.	6123	.
6124.	6124	.
6125.	6125	.
6126.	6126	.
6127.	6127	.
6128.	6128	.
6129.	6129	.
6130.	6130	.
6131.	6131	.
6132.	6132	.
6133.	6133	.
6134.	6134	.
6135.	6135	.
6136.	6136	.
6137.	6137	.
6138.	6138	.
6139.	6139	.

51 . scalar start_forecast = _N - 30 + 1

52 . capture drop ln_forecast

```

53 .
54 . capture drop price_forecast

55 . predict ln_forecast, xb dynamic(start_forecast)

56 . generate price_forecast = exp(ln_forecast)

57 . list time_index price_forecast if missing(close)
    close not found
    r(111);

58 . list time_index price_forecast if missing(Close)

```

	time_i~x	price_~t
6020.	6020	.9998139
6021.	6021	.9998574
6022.	6022	.9998981
6023.	6023	.9999361
6024.	6024	.9999716
6025.	6025	1.000005
6026.	6026	1.000036
6027.	6027	1.000064
6028.	6028	1.000091
6029.	6029	1.000116
6030.	6030	1.00014
6031.	6031	1.000162
6032.	6032	1.000182
6033.	6033	1.000201
6034.	6034	1.000219
6035.	6035	1.000236
6036.	6036	1.000251
6037.	6037	1.000266
6038.	6038	1.000279
6039.	6039	1.000292
6040.	6040	1.000304
6041.	6041	1.000315
6042.	6042	1.000325
6043.	6043	1.000335
6044.	6044	1.000344
6045.	6045	1.000352
6046.	6046	1.00036
6047.	6047	1.000367
6048.	6048	1.000374
6049.	6049	1.00038
6050.	6050	1.000386
6051.	6051	1.000392
6052.	6052	1.000397
6053.	6053	1.000402
6054.	6054	1.000406
6055.	6055	1.00041
6056.	6056	1.000414
6057.	6057	1.000418
6058.	6058	1.000421
6059.	6059	1.000425
6060.	6060	1.000428
6061.	6061	1.00043
6062.	6062	1.000433

6063.	6063	1.000435
6064.	6064	1.000438
6065.	6065	1.00044
6066.	6066	1.000442
6067.	6067	1.000444
6068.	6068	1.000445
6069.	6069	1.000447
6070.	6070	1.000448
6071.	6071	1.00045
6072.	6072	1.000451
6073.	6073	1.000452
6074.	6074	1.000453
6075.	6075	1.000455
6076.	6076	1.000455
6077.	6077	1.000456
6078.	6078	1.000457
6079.	6079	1.000458
6080.	6080	1.000459
6081.	6081	1.00046
6082.	6082	1.00046
6083.	6083	1.000461
6084.	6084	1.000461
6085.	6085	1.000462
6086.	6086	1.000462
6087.	6087	1.000463
6088.	6088	1.000463
6089.	6089	1.000464
6090.	6090	1.000464
6091.	6091	1.000464
6092.	6092	1.000465
6093.	6093	1.000465
6094.	6094	1.000465
6095.	6095	1.000466
6096.	6096	1.000466
6097.	6097	1.000466
6098.	6098	1.000466
6099.	6099	1.000467
6100.	6100	1.000467
6101.	6101	1.000467
6102.	6102	1.000467
6103.	6103	1.000467
6104.	6104	1.000467
6105.	6105	1.000468
6106.	6106	1.000468
6107.	6107	1.000468
6108.	6108	1.000468
6109.	6109	1.000468
6110.	6110	1.000468
6111.	6111	1.000468
6112.	6112	1.000468
6113.	6113	1.000468
6114.	6114	1.000468
6115.	6115	1.000468
6116.	6116	1.000468
6117.	6117	1.000469
6118.	6118	1.000469

6119.	6119	1.000469
6120.	6120	1.000469
6121.	6121	1.000469
6122.	6122	1.000469
6123.	6123	1.000469
6124.	6124	1.000469
6125.	6125	1.000469
6126.	6126	1.000469
6127.	6127	1.000469
6128.	6128	1.000469
6129.	6129	1.000469
6130.	6130	1.000469
6131.	6131	1.000469
6132.	6132	1.000469
6133.	6133	1.000469
6134.	6134	1.000469
6135.	6135	1.000469
6136.	6136	1.000469
6137.	6137	1.000469
6138.	6138	1.000469
6139.	6139	1.000469

59 . arima ln_price, arima(1,1,1) vce(robust)

```
(setting optimization to BHHH)
Iteration 0: log pseudolikelihood = 18466.982
Iteration 1: log pseudolikelihood = 18469.471
Iteration 2: log pseudolikelihood = 18469.516
Iteration 3: log pseudolikelihood = 18469.517
Iteration 4: log pseudolikelihood = 18469.517
(switching optimization to BFGS)
Iteration 5: log pseudolikelihood = 18469.517
Iteration 6: log pseudolikelihood = 18469.517
Iteration 7: log pseudolikelihood = 18469.517
Iteration 8: log pseudolikelihood = 18469.517
```

ARIMA regression

```
Sample: 2 thru 6019                Number of obs   =      6018
                                Wald chi2(2)         =    1079.20
Log pseudolikelihood = 18469.52    Prob > chi2      =      0.0000
```

D.ln_price	Semirobust		z	P> z	[95% conf. interval]	
Coefficient	std. err.					
ln_price						
_cons	.0004693	.0001206	3.89	0.000	.0002329	.0007057
ARMA						
ar						
L1.	.933507	.0358009	26.07	0.000	.8633385	1.003676
ma						
L1.	-.9448296	.0321546	-29.38	0.000	-1.007851	-.8818078
/sigma	.0112432	.0002149	52.32	0.000	.010822	.0116645

Note: The test of the variance against zero is one sided, and the two-sided confidence interval is truncated at zero.

```
60 . apture drop ln_forecast
    command apture is unrecognized
    r(199);

61 .
62 . capture drop price_forecast

63 . capture drop ln_forecast

64 .
65 .
66 .
67 . r(199);
    command r is unrecognized
    r(199);

68 .
69 .
70 .
71 . .

72 .
73 . . capture drop price_forecast

74 .
75 .
76 . capture drop price_forecast

77 .
78 .
79 .
80 . . capture drop ln_forecast

81 .
82 .
83 . drop if missing(close)
    close not found
    r(111);

84 .
85 . scalar last_obs = _N

86 .
87 . tsappend, add(30)

88 . drop if missing(Close) scalar last_obs = _N
    invalid 'scalar'
    r(198);

89 .
90 . tsappend, add(30)

91 . drop if missing(Close)
    (180 observations deleted)
```

```

92 .
93 . scalar last_obs = _N

94 .
95 . tsappend, add(30)

96 . predict ln_forecast, y dynamic(last_obs + 1)
    (1 missing value generated)

97 . generate price_forecast = exp(ln_forecast)
    (1 missing value generated)

98 . list time_index price_forecast if missing(close) in -30/L
    close not found
    r(111);

99 . list time_index price_forecast if missing(Close) in -30/L

```

	time_i~x	price_~t
6020.	6020	4978.872
6021.	6021	4978.159
6022.	6022	4977.656
6023.	6023	4977.338
6024.	6024	4977.196
6025.	6025	4977.219
6026.	6026	4977.395
6027.	6027	4977.713
6028.	6028	4978.169
6029.	6029	4978.748
6030.	6030	4979.446
6031.	6031	4980.253
6032.	6032	4981.161
6033.	6033	4982.163
6034.	6034	4983.256
6035.	6035	4984.43
6036.	6036	4985.68
6037.	6037	4987.007
6038.	6038	4988.401
6039.	6039	4989.856
6040.	6040	4991.375
6041.	6041	4992.946
6042.	6042	4994.57
6043.	6043	4996.242
6044.	6044	4997.958
6045.	6045	4999.717
6046.	6046	5001.515
6047.	6047	5003.352
6048.	6048	5005.222
6049.	6049	5007.127

```
100 . twoway (line price_forecast time_index if missing(close), lcolor(blue) lwidth(medium)),
    close not found
    r(111);
```

```
101 . twoway (line price_forecast time_index if missing(Close), lcolor(blue) lwidth(medium)),
```

```
102 . twoway (line close stata_date, lcolor(navy) lwidth(medium)) ///
    variable close not found
    r(111);
```

```
103 .
104 .      (line ma_365 stata_date, lcolor(orange) lwidth(small) lpattern(dash)), ///
    ( is not a valid command name
    r(199);
```

```
105 .
106 .      title("Gold Price & 365-Day Moving Average (2002-2026)", size(medium)) ///
    command title is unrecognized
    r(199);
```

```
107 .
108 .      subtitle("Visualizing structural macro trends over 24 years", size(small)) ///
    command subtitle is unrecognized
    r(199);
```

```
109 .
110 .      xtitle("Year") ytitle("Price (USD)") ///
    command xtitle is unrecognized
    r(199);
```

```
111 .
112 .      legend(order(1 "Closing Price" 2 "365-Day MA")) ///
    command legend is unrecognized
    r(199);
```

```
113 .
114 .      graphregion(color(white)) bgcolor(white)
    command graphregion is unrecognized
    r(199);
```

```
115 . twoway (line price_forecast time_index if missing(Close), lcolor(blue) lwidth(medium)), twoway (line price_forecast t
    > lor(blue) lwidth(medium)),
    ) required
    r(100);
```

```
116 .
117 .
118 .
119 . . twoway (line Close stata_date, lcolor(navy) lwidth(medium)) ///
    / / / is not a twoway plot type
    r(198);
```

```
120 .
121 .
122 . twoway (line Close stata_date, lcolor(navy) lwidth(medium)) //
    / / is not a twoway plot type
    r(198);
```

```
123 . twoway (line Close stata_date, lcolor(navy) lwidth(medium))
124 . twoway (line log_return stata_date, lcolor(teal) lwidth(vthin)),
125 . twoway (line price_forecast time_index if missing(close), lcolor(blue) lwidth(medium)),
    close not found
    r(111);
126 . twoway (line price_forecast time_index if missing(Close), lcolor(blue) lwidth(medium)),
127 . pwd
    C:\Program Files\Stata17
128 .
```